

DS701 Quiz 1 Reference Sheet

Lectures 01–07 · Foundations, clustering, probabilistic modeling, generalization

01 · What is Data Science?

Data science = drawing *defensible conclusions and decisions* from data: **statistical reasoning** + **computation** + **domain understanding**. *Statistics*: learning from data under uncertainty. *ML*: algorithms that improve with data. *AI*: broadest umbrella. *DS*: the practice using all three, acquisition → communication.

Question types (Leek & Peng) — classify *before* choosing a technique:

- *Descriptive*: what does the data say, summarized?
- *Exploratory*: what patterns are worth a closer look?
- *Inferential*: does the pattern hold beyond this sample?
- *Predictive*: what will happen for a new case?
- *Causal*: what happens if we *intervene*?
- *Mechanistic*: exactly how does the effect operate?

Most failures are *type errors* (e.g. a causal question answered with a predictive model): prediction is judged by held-out accuracy, a causal claim by control of confounding.

Project lifecycle (a loop): 1 Frame the question (classify it; define “good enough” first) → 2 Acquire data → 3 Clean & explore (plots before models) → 4 Model (simplest technique for the question type) → 5 Validate (held-out evaluation, sanity checks) → 6 Communicate & deploy.

Two cultures (Breiman). *Model to understand*: interpretable parameters, inference/effects (regression, causal graphs). *Model to predict*: held-out accuracy is all (random forests, neural nets).

Techniques map: group without labels → k-means, hierarchical, GMM · predict a category → trees/forests, k-NN, logistic regression · predict a number → linear regression · compress/visualize → SVD, PCA, t-SNE · “did X cause Y?” → causal graphs · raw signal → neural nets · recommend → matrix factorization · connected data → graphs.

02 · Distances and Similarity

Feature space: object with d features = point in $\mathbb{R}^d$; data matrix X : m objects (rows) $\times$ n features (cols). **One-hot** encode categoricals (house → [1,0,0]); integer codes falsely imply order.

Metric $d(x, y)$: $d(x, x) = 0$; $d(x, y) > 0$ for $x \neq y$; $d(x, y) = d(y, x)$; $d(x, y) \leq d(x, z) + d(z, y)$ (triangle inequality).

Norm $p(\mathbf{v})$: $p(a\mathbf{v}) = |a|p(\mathbf{v})$; $p(\mathbf{u} + \mathbf{v}) \leq p(\mathbf{u}) + p(\mathbf{v})$; $p(\mathbf{v}) = 0 \iff \mathbf{v} = \mathbf{0}$. Every norm induces a metric $d(\mathbf{x}, \mathbf{y}) = p(\mathbf{x} - \mathbf{y})$; not every metric comes from a norm (discrete metric $d = \mathbb{1}[x \neq y]$).

ℓ_p **norm** ($p \geq 1$) → **Minkowski distance**: $\|\mathbf{x} - \mathbf{y}\|_p = (\sum_i |x_i - y_i|^p)^{1/p}$.

- ℓ_2 **Euclidean** $\sqrt{\sum_i (x_i - y_i)^2}$ — smooth, differentiable.
- ℓ_1 **Manhattan** $\sum_i |x_i - y_i|$ (bit vectors: **Hamming** = # bit flips = size of set difference) — sparse data, robust to outliers.
- ℓ_∞ **Chebyshev** / **max** $\max_i |x_i - y_i|$ — uniform bounds.
- Small p : many small differences count; large p : the largest difference dominates.

Cosine similarity $\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$: ≈ 1 same direction, 0 orthogonal; ignores magnitude (document length). $1 - \cos$

is a common dissimilarity but **not a metric**.

Jaccard (IoU) for sets/bit vectors: $J_{\text{Sim}} = \frac{|\mathbf{x} \cap \mathbf{y}|}{|\mathbf{x} \cup \mathbf{y}|} \in [0, 1]$; $J_{\text{Dist}} = 1 - J_{\text{Sim}}$ is a metric. Prefer over Hamming when set sizes differ (normalized intersection). (*Time-series distances: deferred to a later module.*)

03 · Clustering I: k-means

Clustering: group objects so those within a group are similar (near) and dissimilar (far) from other groups — minimize intra-cluster, maximize inter-cluster distance. **Unsupervised** (no labels) vs **supervised** (learn a label function). **Partitional**: each object in exactly one cluster (mutually exclusive, exhaustive); K given.

k-means objective (WCSS / inertia): choose clusters C_k with centers c_k to minimize $\sum_{k=1}^K \sum_{x \in C_k} \|x - c_k\|^2$. NP-hard for $d \geq 2$; Lloyd’s algorithm is a heuristic.

Lloyd’s algorithm:

1. Pick K centers $c_1, \dots, c_K$ (random data points, or k -means++).
2. Assign each point to its **closest center** → clusters C_j .
3. Update $c_j \leftarrow$ mean (center of mass) of C_j .
4. Repeat 2–3 until convergence: centers move $<$ threshold, inertia change $<$ threshold, or max iterations.

Limitations: implicitly assumes spherical, isotropic, equal-size, equal-variance clusters; sensitive to initialization (can get stuck in a bad solution) and to K . **k-means++**: pick initial centers far apart (**sklearn** default). K is a **hyperparameter** (set before parameters are learned): sweep K , use a criterion (e.g. **elbow** in WCSS vs K). **Feature scaling**: unscaled Euclidean distance is dominated by the largest-range feature; standardize each feature — subtract mean, divide by std.

04 · Clustering II: Hierarchical

Hierarchical clustering = nested clusters in a tree: structure at multiple scales, with containment. **Dendrogram**: leaves = points (x-axis), merge height = inter-cluster distance (y-axis); **cut at a level** → flat clustering with any number of clusters, decided afterwards (e.g. peak **silhouette score**).

Agglomerative (bottom-up, most common): singletons → repeatedly merge the two closest clusters → one cluster.

Divisive (top-down): one cluster → repeatedly split → singletons.

Inputs: data matrix or condensed distance matrix; **linkage**; **metric**. **Output**: $(n-1) \times 4$ **linkage matrix**, one merge per row [idx1, idx2, dist, count]; indices $0..n-1$ = points, $n, n+1, \dots$ = new clusters in creation order.

Linkage (cluster distance):

- **Single**: $D = \min\{d(x, y) : x \in C_i, y \in C_j\}$ — handles non-elliptical shapes and unequal sizes; sensitive to noise/outliers.
- **Complete**: $D = \max\{d(x, y) : x \in C_i, y \in C_j\}$ — balanced diameters, less noise-sensitive; outlier-sensitive, compact/spherical.
- **Average**: $D = \frac{1}{|C_i||C_j|} \sum_{x \in C_i, y \in C_j} d(x, y)$ — compromise; noise-robust, biased to elliptical.
- **Ward**: increase in WCSS from merging, $D = \sum_{C_i} \|x - c_i\|^2 + \sum_{C_j} \|x - c_j\|^2 - \sum_{C_{i+j}} \|x - c_{i+j}\|^2$ — hierarchical analog of k-means.

vs k-means: k-means gives spherical, equal-size, non-overlapping clusters with K fixed; hierarchical encodes many

clusterings at once, works with similarity or distance, and the tree may be meaningful (taxonomy).

05 · Probabilistic Modeling

Recipe: (1) look at data, pick a family whose shape matches; (2) estimate parameters; (3) check the fit (overlay density, predicted vs observed counts, implied properties); (4) use it.

Gaussian $X \sim \mathcal{N}(\mu, \sigma^2)$: $p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}(\frac{x-\mu}{\sigma})^2}$; $\approx 68\%$

within $\mu \pm \sigma$, $\approx 95\%$ within $\mu \pm 2\sigma$.

Poisson (counts at constant rate $\lambda > 0$): $P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$, $k = 0, 1, \dots$; mean = var = λ ; $\hat{\lambda}$ = sample mean count; expected # of N trials with count $k = N p(k)$; var $\gg$ mean (overdispersion) $\Rightarrow$ wrong model.

MLE. i.i.d. $x_1 \dots x_N$: likelihood $L(\theta) = \prod_n p(x_n | \theta)$; log-likelihood $\ell = \sum_n \log p(x_n | \theta)$ (log monotone, product $\rightarrow$ sum); $\hat{\theta} = \arg \max \ell$ via $\nabla_{\theta} \ell = 0$. Gaussian: $\ell = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_n (x_n - \mu)^2 \Rightarrow \hat{\mu} = \frac{1}{N} \sum_n x_n$, $\hat{\sigma}^2 = \frac{1}{N} \sum_n (x_n - \hat{\mu})^2$ (MLE divides by N ; sample std s by $N-1$).

Poisson: same recipe gives $\hat{\lambda} = \frac{1}{N} \sum_n k_n$.

Covariance / correlation: $\text{Cov}(X, Y) = \sigma_{XY} = E[(X - \mu_X)(Y - \mu_Y)]$; $\text{Cov}(X, X) = \sigma_X^2$; $\rho(X, Y) = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \in [-1, 1]$ (Pearson r). $\rho = 0$ = uncorrelated (no linear relation); independent $\Rightarrow$ uncorrelated, not conversely.

Multivariate Gaussian $\mathbf{X} \sim \mathcal{N}(\mu, \Sigma)$: $\mu = E[\mathbf{X}]$, $\Sigma_{ij} = \text{Cov}(X_i, X_j)$; MLE = sample mean vector & sample covariance matrix; Σ sets the shape: circles if $\Sigma = I$, tilted ellipses if $\Sigma_{12} \neq 0$.

CLT: the sum/average of many independent observations from any finite-variance distribution tends to Gaussian. n samples, mean μ , var σ^2 : $\bar{x} \sim \mathcal{N}(\mu, \sigma^2/n)$ (rule of thumb $n \geq 30$). **Standard error** = $\sigma/\sqrt{n}$.

Confidence interval for a mean. Level γ : (A, B) with $P(A < \mu < B) \geq \gamma$; μ fixed, interval random. $z_{1-\alpha/2} = 1 - \alpha/2$ quantile of $\mathcal{N}(0, 1)$ ($z_{0.95} \approx 1.645$, $z_{0.975} \approx 1.96$):

$$\bar{x} \pm z_{1-\alpha/2} \frac{s}{\sqrt{n}}, \quad s = \sqrt{\frac{1}{n-1} \sum_i (x_i - \bar{x})^2}.$$

95% CI $\approx \bar{x} \pm 2$ SE. The level describes the *procedure* ($\approx 95\%$ of such intervals cover μ); width $\propto 1/\sqrt{n}$; uncertainty in the mean, not individual spread; assumes independent samples.

06 · Clustering III: GMM and EM

Soft clustering: x belongs to cluster k with probability $P(C_k | x)$, $\sum_k P(C_k | x) = 1$ (vs k -means hard assignment).

Bayes' rule: $P(C_k | x) = \frac{P(x | C_k)P(C_k)}{\sum_j P(x | C_j)P(C_j)}$.

Multivariate Gaussian pdf ($\mathbf{x} \in \mathbb{R}^d$):

$$\mathcal{N}(\mathbf{x} | \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)\right)$$

GMM (generative): choose component k w.p. π_k ($\sum_k \pi_k = 1$), draw $x \sim \mathcal{N}(\mu_k, \Sigma_k)$; $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$; density and log-likelihood

$$p(x_i | \theta) = \sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k), \quad \ell(\theta) = \sum_{i=1}^n \log p(x_i | \theta).$$

Sum inside the log $\Rightarrow$ no closed-form MLE. Latent z_i = generating cluster; known z would make the MLE trivial (per-cluster mean, covariance, fraction).

EM for GMM:

1. **Init:** μ_k random or from k -means; $\Sigma_k = I$ (or sample cov); $\pi_k = 1/K$.
2. **E-step** — responsibilities ($\sum_k \gamma_{ik} = 1$):

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

3. **M-step** — weighted MLEs with $N_k = \sum_i \gamma_{ik}$: $\mu_k = \frac{1}{N_k} \sum_i \gamma_{ik} x_i$, $\pi_k = \frac{N_k}{n}$,

$$\Sigma_k = \frac{1}{N_k} \sum_i \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^T$$

4. **Convergence:** $|\ell^{(t)} - \ell^{(t-1)}| < \text{tol}$, or $\|\theta^{(t)} - \theta^{(t-1)}\|_2 < \text{tol}$, or $t > \text{max iterations}$.

Log-likelihood never decreases; converges to a **local** maximum $\Rightarrow$ init-dependent; run several starts. **Covariance types / # params:** full Kd^2 · tied (shared Σ) d^2 · diag (axis-aligned) Kd · spherical K . **vs k -means:** same loop (init $\rightarrow$ assign $\rightarrow$ update $\rightarrow$ repeat) with soft assignments (“soft k -means”); GMM makes its generative assumption explicit; captures non-spherical, overlapping clusters.

07 · Generalization

Supervised learning: training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ (features $\mathbf{x}$, labels y); learn a rule to predict y_j for **test data** $\{(\mathbf{x}_j, y_j)\}_{j=1}^M$ unseen in training. **Contract:** (1) a set of candidate functions to search; (2) train and test pairs drawn **i.i.d. from the same distribution** (the future looks like the past).

Polynomial fitting: $f(x, \mathbf{w}) = \sum_{j=0}^k w_j x^j$; **least squares** training error $E(\mathbf{w}) = \sum_{n=1}^N [f(x_n, \mathbf{w}) - y_n]^2$. **w = parameters** (learned from training data); k = **hyperparameter** (set first; controls complexity). Choosing k = **model selection**.

Generalization = performance on unseen data. **Overfitting:** as complexity grows, training error keeps falling (to 0 when #params $\geq$ #points) while test error rises past a minimum. Causes: too many parameters for the data; model more complex than the phenomenon (fits noise). **Remedies:** more training data · limit complexity via hyperparameters · regularization.

Bias: error from an overly simple model — high bias $\Rightarrow$ **underfits**. **Variance:** error from an overly complex model — high variance $\Rightarrow$ **overfits** (a different training sample gives a very different fit). **Trade-off:** flexibility lowers bias, raises variance; test error is U-shaped in complexity; choose the minimum-total-error complexity.

Hold-out protocol: never pick hyperparameters by training error. Randomly split train/test; fit on train, evaluate on test.

Grid search: for each candidate hyperparameter value: split randomly (e.g. 5 $\times$), fit, test, average train & test error; take the lowest mean test error. **Random subsampling:** repeat random partitions (5 or more). **K -fold CV:** partition once into k folds; each fold is the test set once, train on the other $k-1$; average. Larger k : more training data per fit, more fits; $k = N$ = **leave-one-out**. Hyperparameters are always chosen by cross-validation, never training error.